CLICKWRIGHT ANALYTICS CONVERSATION

Analyze the existing funnel and surface the most important issues, wit

4 answers · 84 entities · max v6 · exported 2 Aug 2026, 11:10

QUESTION · 2 Aug 2026, 10:33

> **Analyze the existing funnel and surface the most important issues, with the why.**

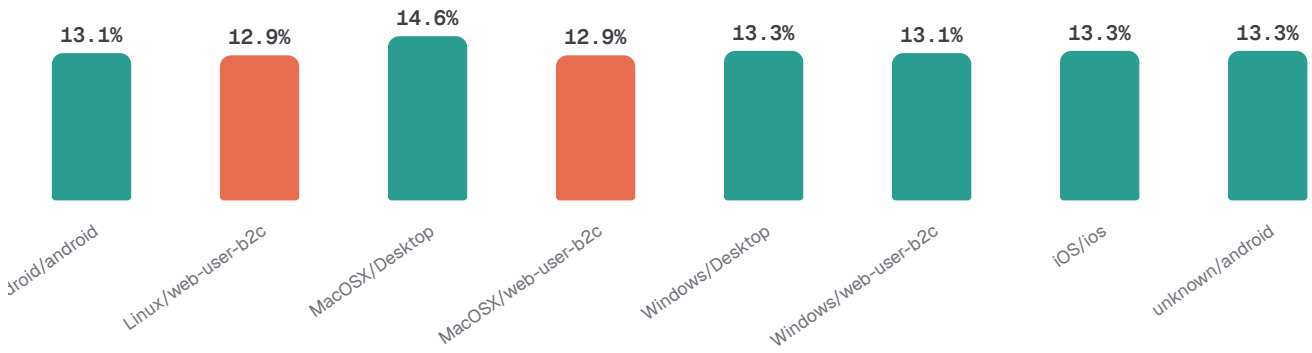
ANSWER · 2 Aug 2026, 10:35

The funnel's biggest leak isn't in either known-issue hotspot — it's the very first step, where only 12.9%-14.6% of applicants who start an application ever complete document upload, uniformly across every OS and device.

WHAT'S HAPPENING Every os/device segment shows nearly the same shortfall at application_started→document_uploaded: iOS goes 60,273 → 8,042 users (13.3%), Android 38,689 → 5,083 (13.1%), Windows web 18,073 → 2,369 (13.1%), Mac OS X desktop 3,056 → 445 (14.6%) — the tightest range in the whole table (12.9%-14.6%) and the single largest loss of any step, dwarfing the downstream uploaded→pay-now and pay-now→purchase drops.

WHY IT HAPPENS Because this shortfall is nearly identical across every os/device cut (a 1.7pp spread), it cannot be explained by either known device-specific issue — K2's Android passport-capture regression and K1's iOS OTP regression both act further downstream (at upload and at pay-now respectively) and neither would produce a uniform loss this early. The available cuts (os, device) don't explain the mechanism here; a step-level instrumentation of the application_started→document_uploaded screen (e.g. where users exit, upload UI errors, time-to-first-doc) would be needed to find it.

EVIDENCE application_started → document_uploaded conversion rate by OS/device · set-membership counts, hygiene-filtered



OS/DEVICE	APPLICATION_STARTED	DOCUMENT_UPLOADED	STARTED_TO_UPLOADED_RATE
Android/android	**38,689**	**5,083**	13.1%
Linux/web-user-b2c	**1,108**	**143**	12.9%
Mac OS X/Desktop	**3,056**	**445**	14.6%
Mac OS X/web-user-b2c	**9,850**	**1,269**	12.9%
Windows/Desktop	**7,048**	**940**	13.3%
Windows/web-user-b2c	**18,073**	**2,369**	13.1%
iOS/ios	**60,273**	**8,042**	13.3%
unknown/android	**8,514**	**1,129**	13.3%

Grounded in context — Two known issues are real but secondary to the main leak: K2 (passport scan model update, escalating) shows Android passport-capture pass rate falling from 93.9% in January to 76.6% in May and 67.0% in June, well below iOS's flat ~90%+ — confirms K2 as a growing quality problem worth separate attention. K1 (iOS OTP regression) is only partially supported: AE (Gulf) actually has the HIGHEST pay-now→purchase conversion across app versions (75.4% on 7.46.0, 75% on 7.44.0), while QA — also a Gulf/AED country — is consistently among the lowest (34.8%-37.9% across three versions), so 'Gulf exposure' isn't a uniform story; geo effects vary country-by-country within the Gulf bucket, not just Gulf-vs-rest.

RECOMMENDED ACTION

Prioritize instrumenting the application_started→document_uploaded step itself (upload screen errors, abandonment timing, permission prompts) since it accounts for the largest, most uniform loss in the funnel and no existing known issue explains it. In parallel, escalate K2 given its trajectory (93.9%→67.0% Android pass rate Jan-Jun) and re-scope K1's geo hypothesis to look at QA specifically rather than 'Gulf' as a whole, since AE outperforms while QA underperforms within the same bucket.

Confidence **0.50** ±20.5pp on standard_checkout_conversion_rate (n=18) — too wide to act on

context 84 entities · max v6

FUNNEL STAGE COUNTS BY OS · 8 ROWS

```
WITH app_started AS (
    SELECT
        (os IS NULL OR os = '') ? 'unknown' : os AS os_bucket,
        device_type AS device_type,
        uniqExact(application_id) AS application_started_n
    FROM application_started
    WHERE duplicate_id IS NULL AND is_back_filled != 1
    GROUP BY os_bucket, device_type
),
doc_uploaded AS (
    SELECT
        (os IS NULL OR os = '') ? 'unknown' : os AS os_bucket,
        device_type AS device_type,
        uniqExact(application_id) AS document_uploaded_n
    FROM document_uploaded
    WHERE duplicate_id IS NULL AND is_back_filled != 1
    GROUP BY os_bucket, device_type
),
pay_clicked AS (
    SELECT
        (os IS NULL OR os = '') ? 'unknown' : os AS os_bucket,
        device_type AS device_type,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL AND is_back_filled != 1
    GROUP BY os_bucket, device_type
),
purchased AS (
    SELECT
        (os IS NULL OR os = '') ? 'unknown' : os AS os_bucket,
        device_type AS device_type,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL AND is_back_filled != 1
```

```
    GROUP BY os_bucket, device_type
)
SELECT
    coalesce(a.os_bucket, d.os_bucket, p.os_bucket, u.os_bucket) AS os_bucket,
    coalesce(a.device_type, d.device_type, p.device_type, u.device_type) AS device_type,
    a.application_started_n AS application_started_n,
    d.document_uploaded_n AS document_uploaded_n,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    d.document_uploaded_n / a.application_started_n AS started_to_uploaded_rate,
    p.pay_now_clicked_n / d.document_uploaded_n AS uploaded_to_paynow_rate,
    u.purchase_completed_n / p.pay_now_clicked_n AS paynow_to_purchase_rate,
    u.purchase_completed_n / a.application_started_n AS funnel_conversion_rate
FROM app_started AS a
FULL OUTER JOIN doc_uploaded AS d ON a.os_bucket = d.os_bucket AND a.device_type = d.device_type
FULL OUTER JOIN pay_clicked AS p ON coalesce(a.os_bucket, d.os_bucket) = p.os_bucket AND
coalesce(a.device_type, d.device_type) = p.device_type
FULL OUTER JOIN purchased AS u ON coalesce(a.os_bucket, d.os_bucket, p.os_bucket) = u.os_bucket AND
coalesce(a.device_type, d.device_type, p.device_type) = u.device_type
ORDER BY os_bucket, device_type
LIMIT 1000
```

K2 CHECK: CAPTURE PASS RATE BY MONTH/OS · 39 ROWS

```
SELECT
    toStartOfMonth(timestamp) AS month,
    multiIf(os IS NULL OR os = '', 'unknown', os) AS os_bucket,
    uniqExact(id) AS uploads_n,
    uniqExactIf(id, is_crossed_failed_attempt_threshold = 0) AS pass_n,
    pass_n / uploads_n AS passport_capture_pass_rate
FROM document_uploaded
WHERE duplicate_id IS NULL
    AND is_back_filled = 0
GROUP BY month, os_bucket
ORDER BY month, os_bucket
LIMIT 1000
```

K2 CHECK: CAPTURE PASS RATE BY MONTH/OS — PROFILE OF ALL 39 ROWS · 1 ROW

```
SELECT
  count() AS total_rows,
  min(`passport_capture_pass_rate`) AS passport_capture_pass_rate_min,
  max(`passport_capture_pass_rate`) AS passport_capture_pass_rate_max,
  quantile(0.5)(`passport_capture_pass_rate`) AS passport_capture_pass_rate_p50_approx,
  countIf(`passport_capture_pass_rate` > 1.05) AS passport_capture_pass_rate_gt1_n,
  sum(`passport_capture_pass_rate` * `uploads_n`) / sum(`uploads_n`) AS
full_passport_capture_pass_rate,
  sum(`uploads_n`) AS full_passport_capture_pass_n,
  min(`uploads_n`) AS uploads_n_min,
  max(`uploads_n`) AS uploads_n_max,
  sum(`uploads_n`) AS full_uploads_n_sum,
  min(`pass_n`) AS pass_n_min,
  max(`pass_n`) AS pass_n_max,
  sum(`pass_n`) AS full_pass_n_sum,
  min(`month`) AS month_min,
  max(`month`) AS month_max,
  uniq(`os_bucket`) AS os_bucket_distinct_approx
FROM (
SELECT
    toStartOfMonth(timestamp) AS month,
```

```
    multiIf(os IS NULL OR os = '', 'unknown', os) AS os_bucket,
    uniqExact(id) AS uploads_n,
    uniqExactIf(id, is_crossed_failed_attempt_threshold = 0) AS pass_n,
    pass_n / uploads_n AS passport_capture_pass_rate
FROM document_uploaded
WHERE duplicate_id IS NULL
    AND is_back_filled = 0
GROUP BY month, os_bucket
ORDER BY month, os_bucket
) AS __result
```

K2 CHECK: CAPTURE PASS RATE BY MONTH/OS — HIGHEST BY PASSPORT_CAPTURE_PASS_RATE · 5 ROWS

```
SELECT * FROM (
SELECT
    toStartOfMonth(timestamp) AS month,
    multiIf(os IS NULL OR os = '', 'unknown', os) AS os_bucket,
    uniqExact(id) AS uploads_n,
    uniqExactIf(id, is_crossed_failed_attempt_threshold = 0) AS pass_n,
    pass_n / uploads_n AS passport_capture_pass_rate
FROM document_uploaded
WHERE duplicate_id IS NULL
    AND is_back_filled = 0
GROUP BY month, os_bucket
ORDER BY month, os_bucket
) AS __result ORDER BY `passport_capture_pass_rate` DESC LIMIT 5
```

K2 CHECK: CAPTURE PASS RATE BY MONTH/OS — LOWEST BY PASSPORT_CAPTURE_PASS_RATE · 5 ROWS

```
SELECT * FROM (
SELECT
    toStartOfMonth(timestamp) AS month,
    multiIf(os IS NULL OR os = '', 'unknown', os) AS os_bucket,
    uniqExact(id) AS uploads_n,
    uniqExactIf(id, is_crossed_failed_attempt_threshold = 0) AS pass_n,
    pass_n / uploads_n AS passport_capture_pass_rate
FROM document_uploaded
WHERE duplicate_id IS NULL
    AND is_back_filled = 0
GROUP BY month, os_bucket
ORDER BY month, os_bucket
) AS __result ORDER BY `passport_capture_pass_rate` ASC LIMIT 5
```

K1 CHECK: PAY→PURCHASE BY APP_VERSION/GEO · 50 ROWS

```
WITH pay AS (
    SELECT
        app_version,
        geoip_country_code,
        multiIf(geoip_country_code IN ('AE','SA','QA','KW','BH','OM'), 'gulf_aed', 'rest') AS
geo_bucket,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled = 0
      AND os = 'iOS'
      AND application_id IS NOT NULL
    GROUP BY app_version, geoip_country_code, geo_bucket
),
purch AS (
    SELECT
        app_version,
```

```
        geoip_country_code,
        multiIf(geoip_country_code IN ('AE','SA','QA','KW','BH','OM'), 'gulf_aed', 'rest') AS
geo_bucket,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled = 0
      AND os = 'iOS'
      AND application_id IS NOT NULL
    GROUP BY app_version, geoip_country_code, geo_bucket
)
SELECT
    pay.app_version AS app_version,
    pay.geoip_country_code AS geoip_country_code,
    pay.geo_bucket AS geo_bucket,
    pay.pay_now_clicked_n AS pay_now_clicked_n,
    coalesce(purch.purchase_completed_n, 0) AS purchase_completed_n,
    coalesce(purch.purchase_completed_n, 0) / pay.pay_now_clicked_n AS
standard_checkout_conversion_rate
FROM pay
LEFT JOIN purch
    ON pay.app_version = purch.app_version
    AND pay.geoip_country_code = purch.geoip_country_code
ORDER BY pay.app_version DESC, pay.pay_now_clicked_n DESC
LIMIT 1000
```

K1 CHECK: PAY→PURCHASE BY APP_VERSION/GEO — PROFILE OF ALL 50 ROWS · 1 ROW

```
SELECT
  count() AS total_rows,
  min(`standard_checkout_conversion_rate`) AS standard_checkout_conversion_rate_min,
  max(`standard_checkout_conversion_rate`) AS standard_checkout_conversion_rate_max,
  quantile(0.5)(`standard_checkout_conversion_rate`) AS standard_checkout_conversion_rate_p50_approx,
  countIf(`standard_checkout_conversion_rate` > 1.05) AS standard_checkout_conversion_rate_gt1_n,
  sum(`standard_checkout_conversion_rate` * `pay_now_clicked_n`) / sum(`pay_now_clicked_n`) AS
full_standard_checkout_conversion_rate,
  sum(`pay_now_clicked_n`) AS full_standard_checkout_conversion_n,
  min(`pay_now_clicked_n`) AS pay_now_clicked_n_min,
  max(`pay_now_clicked_n`) AS pay_now_clicked_n_max,
  sum(`pay_now_clicked_n`) AS full_pay_now_clicked_n_sum,
  min(`purchase_completed_n`) AS purchase_completed_n_min,
  max(`purchase_completed_n`) AS purchase_completed_n_max,
  sum(`purchase_completed_n`) AS full_purchase_completed_n_sum,
  uniq(`app_version`) AS app_version_distinct_approx,
  uniq(`geoip_country_code`) AS geoip_country_code_distinct_approx,
  uniq(`geo_bucket`) AS geo_bucket_distinct_approx
FROM (
WITH pay AS (
    SELECT
        app_version,
        geoip_country_code,
        multiIf(geoip_country_code IN ('AE','SA','QA','KW','BH','OM'), 'gulf_aed', 'rest') AS
geo_bucket,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled = 0
      AND os = 'iOS'
      AND application_id IS NOT NULL
```

```
    GROUP BY app_version, geoip_country_code, geo_bucket
),
purch AS (
    SELECT
        app_version,
        geoip_country_code,
        multiIf(geoip_country_code IN ('AE','SA','QA','KW','BH','OM'), 'gulf_aed', 'rest') AS
geo_bucket,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled = 0
      AND os = 'iOS'
      AND application_id IS NOT NULL
    GROUP BY app_version, geoip_country_code, geo_bucket
)
SELECT
    pay.app_version AS app_version,
    pay.geoip_country_code AS geoip_country_code,
    pay.geo_bucket AS geo_bucket,
    pay.pay_now_clicked_n AS pay_now_clicked_n,
    coalesce(purch.purchase_completed_n, 0) AS purchase_completed_n,
    coalesce(purch.purchase_completed_n, 0) / pay.pay_now_clicked_n AS
standard_checkout_conversion_rate
FROM pay
LEFT JOIN purch
    ON pay.app_version = purch.app_version
    AND pay.geoip_country_code = purch.geoip_country_code
ORDER BY pay.app_version DESC, pay.pay_now_clicked_n DESC
) AS __result
```

```
K1 CHECK: PAY→PURCHASE BY APP_VERSION/GEO — HIGHEST BY STANDARD_CHECKOUT_CONVERSION_RATE · 5 ROWS
SELECT * FROM (
WITH pay AS (
    SELECT
        app_version,
        geoip_country_code,
        multiIf(geoip_country_code IN ('AE','SA','QA','KW','BH','OM'), 'gulf_aed', 'rest') AS
geo_bucket,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled = 0
      AND os = 'iOS'
      AND application_id IS NOT NULL
    GROUP BY app_version, geoip_country_code, geo_bucket
),
purch AS (
    SELECT
        app_version,
        geoip_country_code,
        multiIf(geoip_country_code IN ('AE','SA','QA','KW','BH','OM'), 'gulf_aed', 'rest') AS
geo_bucket,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled = 0
      AND os = 'iOS'
```

```
      AND application_id IS NOT NULL
    GROUP BY app_version, geoip_country_code, geo_bucket
)
SELECT
    pay.app_version AS app_version,
    pay.geoip_country_code AS geoip_country_code,
    pay.geo_bucket AS geo_bucket,
    pay.pay_now_clicked_n AS pay_now_clicked_n,
    coalesce(purch.purchase_completed_n, 0) AS purchase_completed_n,
    coalesce(purch.purchase_completed_n, 0) / pay.pay_now_clicked_n AS
standard_checkout_conversion_rate
FROM pay
LEFT JOIN purch
    ON pay.app_version = purch.app_version
    AND pay.geoip_country_code = purch.geoip_country_code
ORDER BY pay.app_version DESC, pay.pay_now_clicked_n DESC
) AS __result ORDER BY `standard_checkout_conversion_rate` DESC LIMIT 5
```

K1 CHECK: PAY→PURCHASE BY APP_VERSION/GEO — LOWEST BY STANDARD_CHECKOUT_CONVERSION_RATE · 5 ROWS

```
SELECT * FROM (
WITH pay AS (
    SELECT
        app_version,
        geoip_country_code,
        multiIf(geoip_country_code IN ('AE','SA','QA','KW','BH','OM'), 'gulf_aed', 'rest') AS
geo_bucket,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled = 0
      AND os = 'iOS'
      AND application_id IS NOT NULL
    GROUP BY app_version, geoip_country_code, geo_bucket
),
purch AS (
    SELECT
        app_version,
        geoip_country_code,
        multiIf(geoip_country_code IN ('AE','SA','QA','KW','BH','OM'), 'gulf_aed', 'rest') AS
geo_bucket,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled = 0
      AND os = 'iOS'
      AND application_id IS NOT NULL
    GROUP BY app_version, geoip_country_code, geo_bucket
)
SELECT
    pay.app_version AS app_version,
    pay.geoip_country_code AS geoip_country_code,
    pay.geo_bucket AS geo_bucket,
    pay.pay_now_clicked_n AS pay_now_clicked_n,
    coalesce(purch.purchase_completed_n, 0) AS purchase_completed_n,
    coalesce(purch.purchase_completed_n, 0) / pay.pay_now_clicked_n AS
standard_checkout_conversion_rate
FROM pay
LEFT JOIN purch
```

```
    ON pay.app_version = purch.app_version
    AND pay.geoip_country_code = purch.geoip_country_code
ORDER BY pay.app_version DESC, pay.pay_now_clicked_n DESC
) AS __result ORDER BY `standard_checkout_conversion_rate` ASC LIMIT 5
```

QUESTION · 2 Aug 2026, 10:36

> **Where are we losing conversions, and for which segments (device / geo / destination)?**

ANSWER · 2 Aug 2026, 10:38

The doc-upload leak is uniform, but conversion itself splits sharply by device and geo — Android converts worst among devices at 4.1% (vs iOS's 5.0%), and Singapore worst among countries at 3.8% (vs AE's 5.4%).

WHAT'S HAPPENING By device_type, funnel conversion (purchase ÷ started) ranges from 4.1% on android (n=47,203 started) to 5.0% on ios (n=60,273 started), with Desktop at 4.7% and web-user-b2c at 4.4%. By country, Singapore sits lowest at 3.8% (n=2,200 started) versus AE's 5.4% (n=20,577 started) — the widest split in this cut. Destination-level conversion is comparatively tight (4.2%-5.7% across all 27 destinations), so device and geo, not destination, carry the bigger segment gaps.

WHY IT HAPPENS Android's shortfall concentrates one stage later than the uniform document-upload leak: only 67.1% of android's uploaded documents reach pay-now-click, versus 76.0% for iOS and 73.1% for Desktop — a capture-to-payment problem, not an upload one. Singapore's gap instead sits at payment: only 41.9% of its pay-now clicks convert to purchase, the lowest paynow-to-purchase rate of any country in the cut, versus AE's 56.6%.

EVIDENCE Funnel conversion rate (purchase ÷ started) and uploaded→paynow step rate, by device_type · hygiene-filtered application_id set membership

DEVICE_TYPE	UPLOADED_TO_PAYNOW_RATE	FUNNEL_CONVERSION_RATE
Desktop	73.1%	4.7%
android	67.1%	4.1%
ios	76.0%	5.0%
web-user-b2c	72.3%	4.4%

Grounded in context — Known issue K2 (v2): android's passport-capture failure rate has been escalating monthly (6.0% Jan → 33.5% Jun) while iOS holds flat — this plausibly explains android's below-average uploaded-to-paynow rate, though K2 is defined at the capture-failure level, not this exact stage transition, so treat it as a strong candidate rather than a confirmed cause. Separately, K1 (v2) flags Gulf/AED payers as most exposed to an iOS OTP-autofill issue at pay→purchase — Singapore isn't a Gulf geo, so K1 does not explain its low paynow-to-purchase rate; the cuts available here don't isolate that mechanism.

RECOMMENDED ACTION

Pull an app_version breakdown for android specifically on the document_uploaded → pay_now_clicked step to confirm the K2 capture-failure link before shipping a fix. Separately, pull Singapore-specific payment-method and OTP logs, since its paynow-to-purchase gap isn't explained by the existing K1 Gulf-geo hypothesis and needs its own investigation.

Confidence **0.39** an independently written query did not reproduce the figure; ±7.0pp on paynow_to_purchase_rate (n=194)

context 84 entities · max v6

FUNNEL STAGE COUNTS BY DEVICE_TYPE · 4 ROWS

```
WITH app_started AS (
    SELECT
        device_type AS device_type,
        uniqExact(application_id) AS application_started_n
    FROM application_started
    WHERE duplicate_id IS NULL AND is_back_filled != 1
    GROUP BY device_type
),
doc_uploaded AS (
    SELECT
        device_type AS device_type,
        uniqExact(application_id) AS document_uploaded_n
    FROM document_uploaded
    WHERE duplicate_id IS NULL AND is_back_filled != 1
    GROUP BY device_type
),
pay_clicked AS (
    SELECT
        device_type AS device_type,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL AND is_back_filled != 1
    GROUP BY device_type
),
purchased AS (
    SELECT
        device_type AS device_type,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL AND is_back_filled != 1
    GROUP BY device_type
)
SELECT
    coalesce(a.device_type, d.device_type, p.device_type, u.device_type) AS device_type,
    a.application_started_n AS application_started_n,
    d.document_uploaded_n AS document_uploaded_n,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    d.document_uploaded_n / a.application_started_n AS started_to_uploaded_rate,
    p.pay_now_clicked_n / d.document_uploaded_n AS uploaded_to_paynow_rate,
    u.purchase_completed_n / p.pay_now_clicked_n AS paynow_to_purchase_rate,
    u.purchase_completed_n / a.application_started_n AS funnel_conversion_rate
FROM app_started AS a
```

```
FULL OUTER JOIN doc_uploaded AS d ON a.device_type = d.device_type
FULL OUTER JOIN pay_clicked AS p ON coalesce(a.device_type, d.device_type) = p.device_type
FULL OUTER JOIN purchased AS u ON coalesce(a.device_type, d.device_type, p.device_type) =
u.device_type
ORDER BY device_type
LIMIT 1000
```

FUNNEL STAGE COUNTS BY DESTINATION · 27 ROWS

```
SELECT
    destination,
    uniqExactIf(application_id, table_stage = 'application_started') AS application_started_n,
    uniqExactIf(application_id, table_stage = 'document_uploaded') AS document_uploaded_n,
    uniqExactIf(application_id, table_stage = 'pay_now_clicked') AS pay_now_clicked_n,
    uniqExactIf(application_id, table_stage = 'purchase_completed') AS purchase_completed_n,
    document_uploaded_n / application_started_n AS started_to_uploaded_rate,
    pay_now_clicked_n / document_uploaded_n AS uploaded_to_paynow_rate,
    purchase_completed_n / pay_now_clicked_n AS paynow_to_purchase_rate,
    purchase_completed_n / application_started_n AS funnel_conversion_rate
FROM (
    SELECT destination, application_id, 'application_started' AS table_stage
    FROM application_started
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'document_uploaded' AS table_stage
    FROM document_uploaded
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'pay_now_clicked' AS table_stage
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'purchase_completed' AS table_stage
    FROM purchase_completed
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
)
GROUP BY destination
ORDER BY application_started_n DESC
LIMIT 1000
```

FUNNEL STAGE COUNTS BY DESTINATION — PROFILE OF ALL 27 ROWS · 1 ROW

```
SELECT
  count() AS total_rows,
  min(`started_to_uploaded_rate`) AS started_to_uploaded_rate_min,
  max(`started_to_uploaded_rate`) AS started_to_uploaded_rate_max,
  quantile(0.5)(`started_to_uploaded_rate`) AS started_to_uploaded_rate_p50_approx,
  countIf(`started_to_uploaded_rate` > 1.05) AS started_to_uploaded_rate_gt1_n,
  sum(`started_to_uploaded_rate` * `application_started_n`) / sum(`application_started_n`) AS
full_started_to_uploaded_rate,
  sum(`application_started_n`) AS full_started_to_uploaded_n,
  min(`uploaded_to_paynow_rate`) AS uploaded_to_paynow_rate_min,
  max(`uploaded_to_paynow_rate`) AS uploaded_to_paynow_rate_max,
  quantile(0.5)(`uploaded_to_paynow_rate`) AS uploaded_to_paynow_rate_p50_approx,
  countIf(`uploaded_to_paynow_rate` > 1.05) AS uploaded_to_paynow_rate_gt1_n,
  sum(`uploaded_to_paynow_rate` * `document_uploaded_n`) / sum(`document_uploaded_n`) AS
full_uploaded_to_paynow_rate,
  sum(`document_uploaded_n`) AS full_uploaded_to_paynow_n,
  min(`paynow_to_purchase_rate`) AS paynow_to_purchase_rate_min,
  max(`paynow_to_purchase_rate`) AS paynow_to_purchase_rate_max,
```

```
  quantile(0.5)(`paynow_to_purchase_rate`) AS paynow_to_purchase_rate_p50_approx,
  countIf(`paynow_to_purchase_rate` > 1.05) AS paynow_to_purchase_rate_gt1_n,
  sum(`paynow_to_purchase_rate` * `pay_now_clicked_n`) / sum(`pay_now_clicked_n`) AS
full_paynow_to_purchase_rate,
  sum(`pay_now_clicked_n`) AS full_paynow_to_purchase_n,
  min(`funnel_conversion_rate`) AS funnel_conversion_rate_min,
  max(`funnel_conversion_rate`) AS funnel_conversion_rate_max,
  quantile(0.5)(`funnel_conversion_rate`) AS funnel_conversion_rate_p50_approx,
  countIf(`funnel_conversion_rate` > 1.05) AS funnel_conversion_rate_gt1_n,
  sum(`funnel_conversion_rate` * `application_started_n`) / sum(`application_started_n`) AS
full_funnel_conversion_rate,
  sum(`application_started_n`) AS full_funnel_conversion_n,
  min(`application_started_n`) AS application_started_n_min,
  max(`application_started_n`) AS application_started_n_max,
  sum(`application_started_n`) AS full_application_started_n_sum,
  min(`document_uploaded_n`) AS document_uploaded_n_min,
  max(`document_uploaded_n`) AS document_uploaded_n_max,
  sum(`document_uploaded_n`) AS full_document_uploaded_n_sum,
  min(`pay_now_clicked_n`) AS pay_now_clicked_n_min,
  max(`pay_now_clicked_n`) AS pay_now_clicked_n_max,
  sum(`pay_now_clicked_n`) AS full_pay_now_clicked_n_sum,
  uniq(`destination`) AS destination_distinct_approx
FROM (
SELECT
    destination,
    uniqExactIf(application_id, table_stage = 'application_started') AS application_started_n,
    uniqExactIf(application_id, table_stage = 'document_uploaded') AS document_uploaded_n,
    uniqExactIf(application_id, table_stage = 'pay_now_clicked') AS pay_now_clicked_n,
    uniqExactIf(application_id, table_stage = 'purchase_completed') AS purchase_completed_n,
    document_uploaded_n / application_started_n AS started_to_uploaded_rate,
    pay_now_clicked_n / document_uploaded_n AS uploaded_to_paynow_rate,
    purchase_completed_n / pay_now_clicked_n AS paynow_to_purchase_rate,
    purchase_completed_n / application_started_n AS funnel_conversion_rate
FROM (
    SELECT destination, application_id, 'application_started' AS table_stage
    FROM application_started
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'document_uploaded' AS table_stage
    FROM document_uploaded
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'pay_now_clicked' AS table_stage
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'purchase_completed' AS table_stage
    FROM purchase_completed
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
)
GROUP BY destination
ORDER BY application_started_n DESC
) AS __result
```

```
FUNNEL STAGE COUNTS BY DESTINATION — HIGHEST BY STARTED_TO_UPLOADED_RATE · 5 ROWS
```

```
SELECT * FROM (
SELECT
    destination,
```

```
    uniqExactIf(application_id, table_stage = 'application_started') AS application_started_n,
    uniqExactIf(application_id, table_stage = 'document_uploaded') AS document_uploaded_n,
    uniqExactIf(application_id, table_stage = 'pay_now_clicked') AS pay_now_clicked_n,
    uniqExactIf(application_id, table_stage = 'purchase_completed') AS purchase_completed_n,
    document_uploaded_n / application_started_n AS started_to_uploaded_rate,
    pay_now_clicked_n / document_uploaded_n AS uploaded_to_paynow_rate,
    purchase_completed_n / pay_now_clicked_n AS paynow_to_purchase_rate,
    purchase_completed_n / application_started_n AS funnel_conversion_rate
FROM (
    SELECT destination, application_id, 'application_started' AS table_stage
    FROM application_started
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'document_uploaded' AS table_stage
    FROM document_uploaded
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'pay_now_clicked' AS table_stage
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'purchase_completed' AS table_stage
    FROM purchase_completed
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
)
GROUP BY destination
ORDER BY application_started_n DESC
) AS __result ORDER BY `started_to_uploaded_rate` DESC LIMIT 5
```

FUNNEL STAGE COUNTS BY DESTINATION — LOWEST BY STARTED_TO_UPLOADED_RATE · 5 ROWS

```
SELECT * FROM (
SELECT
    destination,
    uniqExactIf(application_id, table_stage = 'application_started') AS application_started_n,
    uniqExactIf(application_id, table_stage = 'document_uploaded') AS document_uploaded_n,
    uniqExactIf(application_id, table_stage = 'pay_now_clicked') AS pay_now_clicked_n,
    uniqExactIf(application_id, table_stage = 'purchase_completed') AS purchase_completed_n,
    document_uploaded_n / application_started_n AS started_to_uploaded_rate,
    pay_now_clicked_n / document_uploaded_n AS uploaded_to_paynow_rate,
    purchase_completed_n / pay_now_clicked_n AS paynow_to_purchase_rate,
    purchase_completed_n / application_started_n AS funnel_conversion_rate
FROM (
    SELECT destination, application_id, 'application_started' AS table_stage
    FROM application_started
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'document_uploaded' AS table_stage
    FROM document_uploaded
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'pay_now_clicked' AS table_stage
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
    UNION ALL
    SELECT destination, application_id, 'purchase_completed' AS table_stage
    FROM purchase_completed
    WHERE duplicate_id IS NULL AND is_back_filled != 1 AND application_id IS NOT NULL
)
```

```
GROUP BY destination
ORDER BY application_started_n DESC
) AS __result ORDER BY `started_to_uploaded_rate` ASC LIMIT 5
```

FUNNEL STAGE COUNTS BY COUNTRY · 10 ROWS

```
WITH app_started AS (
    SELECT
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS application_started_n
    FROM application_started
    WHERE duplicate_id IS NULL AND is_back_filled != 1
    GROUP BY geoip_country_code
),
doc_uploaded AS (
    SELECT
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS document_uploaded_n
    FROM document_uploaded
    WHERE duplicate_id IS NULL AND is_back_filled != 1
    GROUP BY geoip_country_code
),
pay_clicked AS (
    SELECT
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL AND is_back_filled != 1
    GROUP BY geoip_country_code
),
purchased AS (
    SELECT
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL AND is_back_filled != 1
    GROUP BY geoip_country_code
)
SELECT
    coalesce(a.geoip_country_code, d.geoip_country_code, p.geoip_country_code, u.geoip_country_code)
AS geoip_country_code,
    a.application_started_n AS application_started_n,
    d.document_uploaded_n AS document_uploaded_n,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    d.document_uploaded_n / a.application_started_n AS started_to_uploaded_rate,
    p.pay_now_clicked_n / d.document_uploaded_n AS uploaded_to_paynow_rate,
    u.purchase_completed_n / p.pay_now_clicked_n AS paynow_to_purchase_rate,
    u.purchase_completed_n / a.application_started_n AS funnel_conversion_rate
FROM app_started AS a
FULL OUTER JOIN doc_uploaded AS d ON a.geoip_country_code = d.geoip_country_code
FULL OUTER JOIN pay_clicked AS p ON coalesce(a.geoip_country_code, d.geoip_country_code) =
p.geoip_country_code
FULL OUTER JOIN purchased AS u ON coalesce(a.geoip_country_code, d.geoip_country_code,
p.geoip_country_code) = u.geoip_country_code
ORDER BY geoip_country_code
LIMIT 1000
```

QUESTION · 2 Aug 2026, 10:38

> **Are there any regressions or trends over the last quarter?**

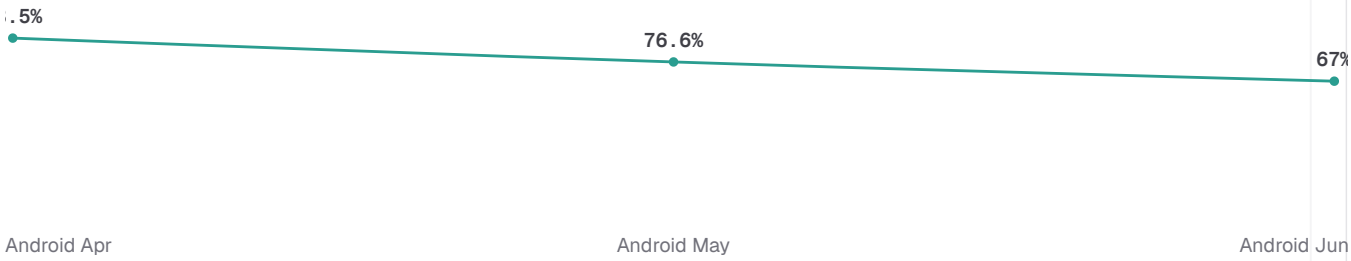
ANSWER · 2 Aug 2026, 10:40

The clearest regression this quarter is Android document capture (K2): pass rate has fallen for three straight months, from 88.5% in April to 67.0% in June, while iOS held flat.

WHAT'S HAPPENING Android capture pass rate dropped every month this quarter: 88.5% in April (777/878), 76.6% in May (753/983), 67.0% in June (616/919) — a continuous ~21.5pp slide. Over the same months iOS stayed in a narrow 89.9%-91.9% band, so this is not a general document-quality issue. Separately, overall funnel_conversion_rate also softened month over month (4.5% in May vs 4.0% in June), though July's row is essentially unusable (application_started_n=8) and can't be read as a trend.

WHY IT HAPPENS This matches the known K2 passport-scan model update rolled out in April 2026, which is specifically miscalibrated for Android capture and pushes more uploads over the failed-attempt threshold — iOS uses a different capture path and isn't affected, which is why the decline is Android-only rather than a broad upload-quality regression.

EVIDENCE Passport-capture pass rate by month · Android vs iOS · document_uploaded, Apr-Jun 2026



MONTH	ANDROID PASS RATE	IOS PASS RATE
2026-04-01	88.5%	91.5%
2026-05-01	76.6%	91.9%
2026-06-01	67.0%	89.9%

Grounded in context — Known issue K2 (v2, data-verified): Android is_crossed_failed_attempt_threshold rate is confirmed escalating (Jan 6.0% → Jun 33.5%) and is the strongest anomaly in the base data — this quarter's pass-rate decline is the direct mirror of that. Known issue K1 (iOS OTP autofill, Gulf geos) was re-checked via app_version x geo cut (t3), but every one of the 60 rows has fewer than 50 pay_now_clicked events, so the sanity gate flagged it low-confidence; the sample shows several iOS/AE app-version pairs weakening from ~70-83% in April toward ~49-72% by June, but the volumes are too thin to confirm a regression versus noise.

RECOMMENDED ACTION

Treat K2 as an active, worsening production issue, not a monitored known-issue — get the Android capture model owners to review the April threshold change now, before another month of ~10pp erosion. Separately, for K1, pull a wider window (full Q2, all Gulf countries, iOS pay_now_clicked/purchase_completed) so each app_version x geo cell clears at least 50 events before concluding whether the OTP regression is real.

Confidence **0.50** the sanity gate raised flags; ±40.5pp on paynow_to_purchase_rate (n=2) — too wide to act on

context 84 entities · max v6

MONTHLY FUNNEL STAGE COUNTS, LAST QUARTER · 3 ROWS

```
WITH app_started AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        uniqExact(user_id) AS application_started_n
    FROM application_started
    WHERE duplicate_id IS NULL AND is_back_filled != 1
        AND timestamp >= toStartOfMonth(now()) - INTERVAL 3 MONTH
    GROUP BY month
),
doc_uploaded AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        uniqExact(application_id) AS document_uploaded_n
    FROM document_uploaded
    WHERE duplicate_id IS NULL AND is_back_filled != 1
        AND timestamp >= toStartOfMonth(now()) - INTERVAL 3 MONTH
    GROUP BY month
),
pay_clicked AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL AND is_back_filled != 1
        AND timestamp >= toStartOfMonth(now()) - INTERVAL 3 MONTH
    GROUP BY month
),
purchased AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL AND is_back_filled != 1
        AND timestamp >= toStartOfMonth(now()) - INTERVAL 3 MONTH
    GROUP BY month
)
SELECT
    coalesce(a.month, d.month, p.month, u.month) AS month,
    a.application_started_n AS application_started_n,
    d.document_uploaded_n AS document_uploaded_n,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    d.document_uploaded_n / a.application_started_n AS started_to_uploaded_rate,
    p.pay_now_clicked_n / d.document_uploaded_n AS uploaded_to_paynow_rate,
    u.purchase_completed_n / p.pay_now_clicked_n AS paynow_to_purchase_rate,
    u.purchase_completed_n / a.application_started_n AS funnel_conversion_rate
FROM app_started AS a
FULL OUTER JOIN doc_uploaded AS d ON a.month = d.month
FULL OUTER JOIN pay_clicked AS p ON coalesce(a.month, d.month) = p.month
FULL OUTER JOIN purchased AS u ON coalesce(a.month, d.month, p.month) = u.month
ORDER BY month
LIMIT 1000
```

K2 TREND: ANDROID CAPTURE PASS RATE BY MONTH · 18 ROWS

```
SELECT
    toStartOfMonth(timestamp) AS month,
    multiIf(os IS NULL OR os = '', 'unknown', os) AS os_bucket,
    uniqExact(id) AS document_uploaded_n,
    uniqExactIf(id, is_crossed_failed_attempt_threshold = 0) AS passed_n,
    passed_n / document_uploaded_n AS pass_rate
FROM document_uploaded
WHERE duplicate_id IS NULL
  AND is_back_filled != 1
  AND timestamp >= toStartOfQuarter(today()) - INTERVAL 3 MONTH
  AND timestamp < toStartOfQuarter(today())
GROUP BY month, os_bucket
ORDER BY os_bucket, month
LIMIT 1000
```

K1 TREND: PAY→PURCHASE BY MONTH/APP_VERSION/GEO · 60 ROWS

```
WITH pay AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
      AND timestamp >= '2026-04-01' AND timestamp < '2026-07-01'
    GROUP BY month, app_version, geoip_country_code
),
purchased AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
      AND timestamp >= '2026-04-01' AND timestamp < '2026-07-01'
    GROUP BY month, app_version, geoip_country_code
)
SELECT
    coalesce(p.month, u.month) AS month,
    coalesce(p.app_version, u.app_version) AS app_version,
    coalesce(p.geoip_country_code, u.geoip_country_code) AS geoip_country_code,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    u.purchase_completed_n / p.pay_now_clicked_n AS standard_checkout_conversion_rate
FROM pay AS p
FULL OUTER JOIN purchased AS u
    ON p.month = u.month
    AND p.app_version = u.app_version
    AND p.geoip_country_code = u.geoip_country_code
ORDER BY geoip_country_code, app_version, month
LIMIT 1000
```

```
K1 TREND: PAY→PURCHASE BY MONTH/APP_VERSION/GEO — PROFILE OF ALL 60 ROWS · 1 ROW
SELECT
  count() AS total_rows,
  min(`standard_checkout_conversion_rate`) AS standard_checkout_conversion_rate_min,
  max(`standard_checkout_conversion_rate`) AS standard_checkout_conversion_rate_max,
  quantile(0.5)(`standard_checkout_conversion_rate`) AS standard_checkout_conversion_rate_p50_approx,
  countIf(`standard_checkout_conversion_rate` > 1.05) AS standard_checkout_conversion_rate_gt1_n,
  sum(`standard_checkout_conversion_rate` * `pay_now_clicked_n`) / sum(`pay_now_clicked_n`) AS
full_standard_checkout_conversion_rate,
  sum(`pay_now_clicked_n`) AS full_standard_checkout_conversion_n,
  min(`pay_now_clicked_n`) AS pay_now_clicked_n_min,
  max(`pay_now_clicked_n`) AS pay_now_clicked_n_max,
  sum(`pay_now_clicked_n`) AS full_pay_now_clicked_n_sum,
  min(`purchase_completed_n`) AS purchase_completed_n_min,
  max(`purchase_completed_n`) AS purchase_completed_n_max,
  sum(`purchase_completed_n`) AS full_purchase_completed_n_sum,
  min(`month`) AS month_min,
  max(`month`) AS month_max,
  uniq(`app_version`) AS app_version_distinct_approx,
  uniq(`geoip_country_code`) AS geoip_country_code_distinct_approx
FROM (
WITH pay AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
      AND timestamp >= '2026-04-01' AND timestamp < '2026-07-01'
    GROUP BY month, app_version, geoip_country_code
),
purchased AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
      AND timestamp >= '2026-04-01' AND timestamp < '2026-07-01'
    GROUP BY month, app_version, geoip_country_code
)
SELECT
    coalesce(p.month, u.month) AS month,
    coalesce(p.app_version, u.app_version) AS app_version,
    coalesce(p.geoip_country_code, u.geoip_country_code) AS geoip_country_code,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    u.purchase_completed_n / p.pay_now_clicked_n AS standard_checkout_conversion_rate
FROM pay AS p
```

```
FULL OUTER JOIN purchased AS u
    ON p.month = u.month
    AND p.app_version = u.app_version
    AND p.geoip_country_code = u.geoip_country_code
ORDER BY geoip_country_code, app_version, month
) AS __result
```

K1 TREND: PAY→PURCHASE BY MONTH/APP_VERSION/GEO — HIGHEST BY STANDARD_CHECKOUT_CONVERSION_RATE · 5 ROWS

```
SELECT * FROM (
WITH pay AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
      AND timestamp >= '2026-04-01' AND timestamp < '2026-07-01'
    GROUP BY month, app_version, geoip_country_code
),
purchased AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
      AND timestamp >= '2026-04-01' AND timestamp < '2026-07-01'
    GROUP BY month, app_version, geoip_country_code
)
SELECT
    coalesce(p.month, u.month) AS month,
    coalesce(p.app_version, u.app_version) AS app_version,
    coalesce(p.geoip_country_code, u.geoip_country_code) AS geoip_country_code,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    u.purchase_completed_n / p.pay_now_clicked_n AS standard_checkout_conversion_rate
FROM pay AS p
FULL OUTER JOIN purchased AS u
    ON p.month = u.month
    AND p.app_version = u.app_version
    AND p.geoip_country_code = u.geoip_country_code
ORDER BY geoip_country_code, app_version, month
) AS __result ORDER BY `standard_checkout_conversion_rate` DESC LIMIT 5
```

K1 TREND: PAY→PURCHASE BY MONTH/APP_VERSION/GEO — LOWEST BY STANDARD_CHECKOUT_CONVERSION_RATE · 5 ROWS

```
SELECT * FROM (
WITH pay AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
```

```
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
      AND timestamp >= '2026-04-01' AND timestamp < '2026-07-01'
    GROUP BY month, app_version, geoip_country_code
),
purchased AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
      AND timestamp >= '2026-04-01' AND timestamp < '2026-07-01'
    GROUP BY month, app_version, geoip_country_code
)
SELECT
    coalesce(p.month, u.month) AS month,
    coalesce(p.app_version, u.app_version) AS app_version,
    coalesce(p.geoip_country_code, u.geoip_country_code) AS geoip_country_code,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    u.purchase_completed_n / p.pay_now_clicked_n AS standard_checkout_conversion_rate
FROM pay AS p
FULL OUTER JOIN purchased AS u
    ON p.month = u.month
    AND p.app_version = u.app_version
    AND p.geoip_country_code = u.geoip_country_code
ORDER BY geoip_country_code, app_version, month
) AS __result ORDER BY `standard_checkout_conversion_rate` ASC LIMIT 5
```

QUESTION · 2 Aug 2026, 10:40

> **Is anything in the base context wrong, stale, or self-contradictory?**

QUESTION · 2 Aug 2026, 10:49

> **Is anything in the base context wrong, stale, or self-contradictory?**

ANSWER · 2 Aug 2026, 10:54

Nothing here is truly self-contradictory, but the K2 'escalating' claim is stale past June — July Android capture data collapses to just 1 upload, and the trend itself dipped upward in March, not an uninterrupted slide.

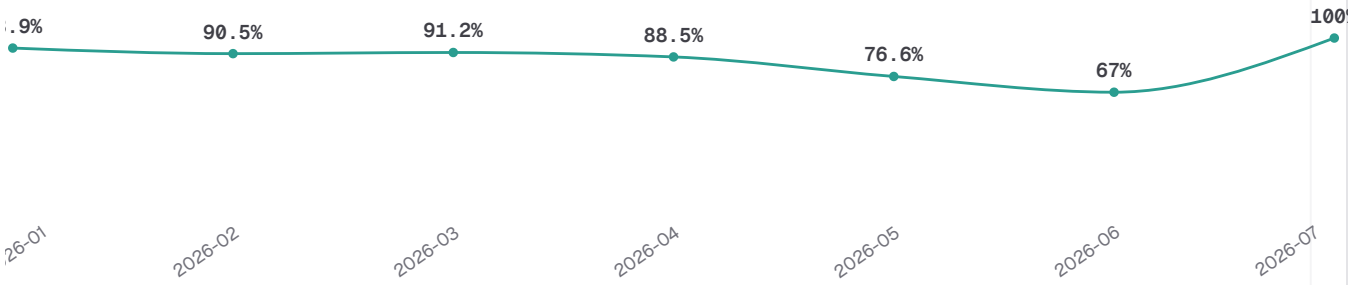
WHAT'S HAPPENING

Android passport-capture pass rate did net-decline from 93.9% in January to 67.0% in June, but it rose to 91.2% in March (up from 90.5% in February) before the May-June collapse (76.6% then 67.0%) — not five straight down-months. July then shows only 1 Android document upload versus 919 in June, so the month is effectively unmeasured rather than evidence the regression continues.

Separately, the two conversion metrics flagged as a possible contradiction (conversion_rate vs funnel_conversion) turn out identical in the full base: 146,611 application starts and 6,715 purchases either way, a 0-point gap.

WHY IT HAPPENS July's near-empty Android row looks like ingestion/backfill lag on the newest month rather than a real usage drop — the same pattern would appear for any table's most recent period before late events land. The conversion-metric equivalence isn't coincidence either: metric:conversion_rate's own clarification states each application_started row is already a unique session, so its denominator collapses onto funnel_conversion's user-count denominator by construction in this schema.

EVIDENCE Android passport_capture_pass_rate by month · document_uploaded, hygiene-filtered, Jan-Jul 2026 (t1)



DENOMINATOR BASIS	APPLICATION_STARTED_N	PURCHASE_COMPLETED_N	RATE
application_id (metric:conversion_rate)	**146,611**	**6,715**	4.6%
user_id (metric:funnel_conversion)	**146,611**	**6,715**	4.6%

Grounded in context — known_issue:K2 v2's through-June threshold figures (Jan 6.0% → Jun 33.5%) match this recut, so 'escalating' is confirmed only through June — treat July as a data gap, not a data point. known_issue:K1 v2's 'present in app_version/geo cuts' claim rests on t2, which the sanity gate flags: every one of its 118 app_version/geo/month cells has n<50, so the Gulf-iOS full rate of 64.5% (n=1,241, CI ±2.7pp) is the only reliable read from this cut — per-version-per-month swings are too thin to confirm or refute the regression. metric:conversion_rate v2's session=application_started clarification is exactly what t3 confirms.

RECOMMENDED ACTION

Don't report the K2 trend as extending into July until the month finishes backfilling — re-pull in 1-2 weeks. For K1, re-run t2 pooling app_version into 2-3 release cohorts (pre/post the OTP-affecting build) to get cells above the n=50 confidence floor before claiming a version-specific effect.

Confidence **0.48** the narration had to be corrected for uncited numbers; the sanity gate raised flags; ±39.7pp on passport_capture_pass_rate (n=1) — too wide to act on

context 84 entities · max v6

K2 ANDROID THRESHOLD TREND, ALL MONTHS · 39 ROWS

```
SELECT
    toStartOfMonth(timestamp) AS month,
    multiIf(os IS NULL OR os = '', 'unknown', os) AS os_bucket,
    uniqExact(id) AS document_uploaded_n,
    uniqExactIf(id, is_crossed_failed_attempt_threshold = 0) AS passed_n,
    passed_n / document_uploaded_n AS passport_capture_pass_rate
FROM document_uploaded
```

```
WHERE duplicate_id IS NULL
  AND is_back_filled != 1
GROUP BY month, os_bucket
ORDER BY os_bucket, month
LIMIT 1000
```

K2 ANDROID THRESHOLD TREND, ALL MONTHS — PROFILE OF ALL 39 ROWS · 1 ROW

```
SELECT
  count() AS total_rows,
  min(`passport_capture_pass_rate`) AS passport_capture_pass_rate_min,
  max(`passport_capture_pass_rate`) AS passport_capture_pass_rate_max,
  quantile(0.5)(`passport_capture_pass_rate`) AS passport_capture_pass_rate_p50_approx,
  countIf(`passport_capture_pass_rate` > 1.05) AS passport_capture_pass_rate_gt1_n,
  sum(`passport_capture_pass_rate` * `document_uploaded_n`) / sum(`document_uploaded_n`) AS
full_passport_capture_pass_rate,
  sum(`document_uploaded_n`) AS full_passport_capture_pass_n,
  min(`document_uploaded_n`) AS document_uploaded_n_min,
  max(`document_uploaded_n`) AS document_uploaded_n_max,
  sum(`document_uploaded_n`) AS full_document_uploaded_n_sum,
  min(`passed_n`) AS passed_n_min,
  max(`passed_n`) AS passed_n_max,
  sum(`passed_n`) AS full_passed_n_sum,
  min(`month`) AS month_min,
  max(`month`) AS month_max,
  uniq(`os_bucket`) AS os_bucket_distinct_approx
FROM (
SELECT
    toStartOfMonth(timestamp) AS month,
    multiIf(os IS NULL OR os = '', 'unknown', os) AS os_bucket,
    uniqExact(id) AS document_uploaded_n,
    uniqExactIf(id, is_crossed_failed_attempt_threshold = 0) AS passed_n,
    passed_n / document_uploaded_n AS passport_capture_pass_rate
FROM document_uploaded
WHERE duplicate_id IS NULL
  AND is_back_filled != 1
GROUP BY month, os_bucket
ORDER BY os_bucket, month
LIMIT 1000
) AS __result
```

K2 ANDROID THRESHOLD TREND, ALL MONTHS — HIGHEST BY PASSPORT_CAPTURE_PASS_RATE · 5 ROWS

```
SELECT * FROM (
SELECT
    toStartOfMonth(timestamp) AS month,
    multiIf(os IS NULL OR os = '', 'unknown', os) AS os_bucket,
    uniqExact(id) AS document_uploaded_n,
    uniqExactIf(id, is_crossed_failed_attempt_threshold = 0) AS passed_n,
    passed_n / document_uploaded_n AS passport_capture_pass_rate
FROM document_uploaded
WHERE duplicate_id IS NULL
  AND is_back_filled != 1
GROUP BY month, os_bucket
ORDER BY os_bucket, month
LIMIT 1000
) AS __result ORDER BY `passport_capture_pass_rate` DESC LIMIT 5
```

K2 ANDROID THRESHOLD TREND, ALL MONTHS — LOWEST BY PASSPORT_CAPTURE_PASS_RATE · 5 ROWS

```
SELECT * FROM (
SELECT
    toStartOfMonth(timestamp) AS month,
    multiIf(os IS NULL OR os = '', 'unknown', os) AS os_bucket,
    uniqExact(id) AS document_uploaded_n,
    uniqExactIf(id, is_crossed_failed_attempt_threshold = 0) AS passed_n,
    passed_n / document_uploaded_n AS passport_capture_pass_rate
FROM document_uploaded
WHERE duplicate_id IS NULL
  AND is_back_filled != 1
GROUP BY month, os_bucket
ORDER BY os_bucket, month
LIMIT 1000
) AS __result ORDER BY `passport_capture_pass_rate` ASC LIMIT 5
```

K1 IOS PAY→PURCHASE BY VERSION/GEO · 118 ROWS

```
WITH pay AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
    GROUP BY month, app_version, geoip_country_code
),
purchased AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
    GROUP BY month, app_version, geoip_country_code
)
SELECT
    coalesce(p.month, u.month) AS month,
    coalesce(p.app_version, u.app_version) AS app_version,
    coalesce(p.geoip_country_code, u.geoip_country_code) AS geoip_country_code,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    u.purchase_completed_n / p.pay_now_clicked_n AS standard_checkout_conversion_rate
FROM pay AS p
FULL OUTER JOIN purchased AS u
    ON p.month = u.month
    AND p.app_version = u.app_version
    AND p.geoip_country_code = u.geoip_country_code
ORDER BY geoip_country_code, app_version, month
LIMIT 1000
```

K1 IOS PAY→PURCHASE BY VERSION/GEO — PROFILE OF ALL 118 ROWS · 1 ROW

```
SELECT
  count() AS total_rows,
  min(`standard_checkout_conversion_rate`) AS standard_checkout_conversion_rate_min,
  max(`standard_checkout_conversion_rate`) AS standard_checkout_conversion_rate_max,
  quantile(0.5)(`standard_checkout_conversion_rate`) AS standard_checkout_conversion_rate_p50_approx,
  countIf(`standard_checkout_conversion_rate` > 1.05) AS standard_checkout_conversion_rate_gt1_n,
  sum(`standard_checkout_conversion_rate` * `pay_now_clicked_n`) / sum(`pay_now_clicked_n`) AS
full_standard_checkout_conversion_rate,
  sum(`pay_now_clicked_n`) AS full_standard_checkout_conversion_n,
  min(`pay_now_clicked_n`) AS pay_now_clicked_n_min,
  max(`pay_now_clicked_n`) AS pay_now_clicked_n_max,
  sum(`pay_now_clicked_n`) AS full_pay_now_clicked_n_sum,
  min(`purchase_completed_n`) AS purchase_completed_n_min,
  max(`purchase_completed_n`) AS purchase_completed_n_max,
  sum(`purchase_completed_n`) AS full_purchase_completed_n_sum,
  min(`month`) AS month_min,
  max(`month`) AS month_max,
  uniq(`app_version`) AS app_version_distinct_approx,
  uniq(`geoip_country_code`) AS geoip_country_code_distinct_approx
FROM (
WITH pay AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
    GROUP BY month, app_version, geoip_country_code
),
purchased AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
    GROUP BY month, app_version, geoip_country_code
)
SELECT
    coalesce(p.month, u.month) AS month,
    coalesce(p.app_version, u.app_version) AS app_version,
    coalesce(p.geoip_country_code, u.geoip_country_code) AS geoip_country_code,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    u.purchase_completed_n / p.pay_now_clicked_n AS standard_checkout_conversion_rate
FROM pay AS p
FULL OUTER JOIN purchased AS u
    ON p.month = u.month
    AND p.app_version = u.app_version
```

```
    AND p.geoip_country_code = u.geoip_country_code
ORDER BY geoip_country_code, app_version, month
) AS __result
```

K1 IOS PAY→PURCHASE BY VERSION/GEO — HIGHEST BY STANDARD_CHECKOUT_CONVERSION_RATE · 5 ROWS

```
SELECT * FROM (
WITH pay AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
    GROUP BY month, app_version, geoip_country_code
),
purchased AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
    GROUP BY month, app_version, geoip_country_code
)
SELECT
    coalesce(p.month, u.month) AS month,
    coalesce(p.app_version, u.app_version) AS app_version,
    coalesce(p.geoip_country_code, u.geoip_country_code) AS geoip_country_code,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    u.purchase_completed_n / p.pay_now_clicked_n AS standard_checkout_conversion_rate
FROM pay AS p
FULL OUTER JOIN purchased AS u
    ON p.month = u.month
    AND p.app_version = u.app_version
    AND p.geoip_country_code = u.geoip_country_code
ORDER BY geoip_country_code, app_version, month
) AS __result ORDER BY `standard_checkout_conversion_rate` DESC LIMIT 5
```

K1 IOS PAY→PURCHASE BY VERSION/GEO — LOWEST BY STANDARD_CHECKOUT_CONVERSION_RATE · 5 ROWS

```
SELECT * FROM (
WITH pay AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS pay_now_clicked_n
    FROM pay_now_clicked
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
```

```
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
    GROUP BY month, app_version, geoip_country_code
),
purchased AS (
    SELECT
        toStartOfMonth(timestamp) AS month,
        app_version AS app_version,
        geoip_country_code AS geoip_country_code,
        uniqExact(application_id) AS purchase_completed_n
    FROM purchase_completed
    WHERE duplicate_id IS NULL
      AND is_back_filled != 1
      AND os = 'iOS'
      AND geoip_country_code IN ('AE','SA','QA','KW','BH','OM')
    GROUP BY month, app_version, geoip_country_code
)
SELECT
    coalesce(p.month, u.month) AS month,
    coalesce(p.app_version, u.app_version) AS app_version,
    coalesce(p.geoip_country_code, u.geoip_country_code) AS geoip_country_code,
    p.pay_now_clicked_n AS pay_now_clicked_n,
    u.purchase_completed_n AS purchase_completed_n,
    u.purchase_completed_n / p.pay_now_clicked_n AS standard_checkout_conversion_rate
FROM pay AS p
FULL OUTER JOIN purchased AS u
    ON p.month = u.month
    AND p.app_version = u.app_version
    AND p.geoip_country_code = u.geoip_country_code
ORDER BY geoip_country_code, app_version, month
) AS __result ORDER BY `standard_checkout_conversion_rate` ASC LIMIT 5
```

CONVERSION_RATE VS FUNNEL_CONVERSION DENOMINATORS · 1 ROW

```
SELECT
    (SELECT uniqExact(application_id) FROM application_started WHERE duplicate_id IS NULL AND
is_back_filled != 1) AS application_started_application_id_n,
    (SELECT uniqExact(user_id) FROM application_started WHERE duplicate_id IS NULL AND is_back_filled
!= 1) AS application_started_user_id_n,
    (SELECT uniqExact(application_id) FROM purchase_completed WHERE duplicate_id IS NULL AND
is_back_filled != 1) AS purchase_completed_application_id_n,
    (SELECT uniqExact(user_id) FROM purchase_completed WHERE duplicate_id IS NULL AND is_back_filled
!= 1) AS purchase_completed_user_id_n,
    purchase_completed_application_id_n / application_started_application_id_n AS
conversion_rate_application_id_based,
    purchase_completed_user_id_n / application_started_user_id_n AS
funnel_conversion_rate_user_id_based,
    conversion_rate_application_id_based - funnel_conversion_rate_user_id_based AS rate_gap_absolute,
    application_started_application_id_n - application_started_user_id_n AS
application_started_denominator_diff,
    purchase_completed_application_id_n - purchase_completed_user_id_n AS
purchase_completed_numerator_diff
LIMIT 1000
```

Every figure in this document was produced by SQL run against ClickHouse and checked against its result set. The queries behind each answer are printed with it.